that thread has locked the mutex already when it makes the call. This has the potential for deadlock *and* the potential to access a mutex after it has already been destroyed, so it's a nonstarter. It would be rather too restrictive if you couldn't *reliably* interrupt a condition variable wait—you can do almost as well without a special `interruptible_wait()`—so what other options do you have? One option is to put a timeout on the wait; use `wait_for()` rather than `wait()` with a small timeout value (such as 1 ms). This puts an upper limit on how long the thread will have to wait before it sees the interruption (subject to the tick granularity of the clock). If you do this, the waiting thread will see rather more "spurious" wakes resulting from the timeout, but it can't easily be helped. Such an implementation is shown in the next listing, along with the corresponding implementation of `interrupt_flag`.

Listing 9.11. Using a timeout in `interruptible_wait` for `std::condition_variable`

```
class interrupt_flag
{
    std::atomic<bool> flag;
    std::condition_variable* thread_cond;
    std::mutex set_clear_mutex;

public:
    interrupt_flag():
         thread_cond(0)
     {}

    void set()
    {
        flag.store(true,std::memory_order_relaxed);
        std::lock_guard<std::mutex> lk(set_clear_mutex);
        if(thread_cond)
        {
            thread_cond->notify_all();
        }
    }

    bool is_set() const
    {
        return flag.load(std::memory_order_relaxed);
    }

    void set_condition_variable(std::condition_variable& cv)
    {
        std::lock_guard<std::mutex> lk(set_clear_mutex);
```